

cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h`. You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

Notation	Meaning
<Species>	<code>Ion</code> , <code>Alpha</code> , <code>Beam</code>
<MHDField>	<code>Phi</code> , <code>A</code> , <code>dNe</code> , <code>dTe</code>
<PICField>	<code>dPi</code> , <code>dPa</code> , <code>dPb</code>

Data Types and File Paths

Parameter	Allowed Values	Description	Notes	Example
<code>mhdReal</code>	<code>float</code> / <code>double</code>	MHD calculation precision.	MHD precision should be greater than or equal to PIC precision.	<code>using mhdReal = double;</code>
<code>picReal</code>	<code>float</code> / <code>double</code>	PIC calculation precision.	PIC precision should be less than or equal to MHD precision.	<code>using picReal = float;</code>
<code>inputDir</code>	String path	Input file directory.	For a run from scratch, only the <code>MHDEquilibrium</code> file is needed. For a continued run, the <code>MHDContinue</code> and <code>PICContinue</code> files are also needed. They come from the previous run's <code>outputDir/final</code> .	<code>const std::string inputDir = "/path/to/input";</code>
<code>outputDir</code>	String path	Output file directory.		<code>const std::string outputDir = "/path/to/output";</code>
<code>MHDEquilibrium</code>	<code>.bin</code> file name	MHD equilibrium file.	Generated by <code>generateInput2D.m</code> for tokamaks and <code>generateInput3D.m</code> for stellarators.	<code>const std::string MHDEquilibrium = "MHDEquilibrium_384_32.bin";</code>
<code><Species>PhaseSpaceMapping</code>	<code>.bin</code> file name	Phase-space mapping file.	Required only when <code>ifOutputPhaseSpace*</code> is enabled; generated by <code>generatePhaseSpaceMapping2D.m</code> .	<code>const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";</code>

Normalization

Parameter	Allowed Values	Description	Notes	Example
<code>B0</code>	Real number	Magnetic-field normalization scale.		<code>const double B0 = 4.921751144619735;</code>
<code>L0</code>	Real number	Length normalization scale.		<code>const double L0 = 6.595629295925759;</code>
<code>VA0</code>	Real number	Velocity normalization scale.		<code>const double VA0 = 8.864164667700194e+06;</code>
<code>RH00</code>	Real number	Left endpoint of the radial interval.		<code>const double RH00 = 0.08;</code>
<code>RH01</code>	Real number	Right endpoint of the radial interval.		<code>const double RH01 = 0.90;</code>
<code>PSITMAX</code>	Real number	Outermost toroidal magnetic flux divided by 2 π (Wb/rad).		<code>const double PSITMAX = 18.868213504765762;</code>

Grid and Parallel Setup

Parameter	Allowed Values	Description	Notes	Example
<code>hostNums</code>	Integer	Number of MPI processes.		<code>const int hostNums = 4;</code>
<code>devNums</code>	Integer	Number of GPUs used by each MPI process.		<code>const int devNums = 4;</code>
<code>gridNx</code>	Integer	Number of radial grid points.		<code>const int gridNx = 256;</code>
<code>gridNy</code>	Integer	Number of field-aligned grid points.	Must be divisible by <code>hostNums * devNums</code> .	<code>const int gridNy = 32;</code>
<code>gridNz</code>	Integer	Number of toroidal grid points.		<code>const int gridNz = 96;</code>
<code>NFP</code>	Integer	Number of field periods. <code>NFP=1</code> for tokamak; <code>NFP!=1</code> for stellarator.		<code>const int NFP = 1;</code>
<code>ppcNums</code>	Integer	Average number of particles per species in each grid cell.		<code>const int ppcNums = 256;</code>

Toroidal Range and Initial Perturbation

Parameter	Allowed Values	Description	Notes	Example
<code>tubes</code>	Integer	Simulate <code>1/tubes</code> of the toroidal domain.		<code>const int tubes = 6;</code>
<code>leftN</code>	Integer	Lower bound of the retained toroidal mode number in the <code>1/tubes</code> toroidal domain.		<code>const int leftN = 1;</code>
<code>rightN</code>	Integer	Upper bound of the retained toroidal mode number in the <code>1/tubes</code> toroidal domain.	Retained toroidal modes are <code>tubes*[leftN,rightN]</code> , spaced by <code>tubes</code> .	<code>const int rightN = 6;</code>
<code>refinedTimes</code>	Integer	Poloidal-grid refinement factor for <code>selectNM_*</code> filtering.	The refined poloidal grid size is <code>gridNy * refinedTimes</code> .	<code>const int refinedTimes = 32;</code>
<code>perturbLeftN</code>	Integer	Lower bound of the initial perturbation toroidal mode number in the <code>1/tubes</code> toroidal domain.		<code>const int perturbLeftN = 1;</code>
<code>perturbRightN</code>	Integer	Upper bound of the initial perturbation toroidal mode number in the <code>1/tubes</code> toroidal domain.	Initial perturbation modes are <code>tubes*[perturbLeftN,perturbRightN]</code> , spaced by <code>tubes</code> .	<code>const int perturbRightN = 6;</code>
<code>perturbRadialIndex</code>	Integer	Radial peak grid point of the initial Gaussian perturbation.		<code>const int perturbRadialIndex = 85;</code>
<code>perturbWidth</code>	Real number	Radial width of the initial Gaussian perturbation.		<code>const mhdReal perturbWidth = 0.04;</code>
<code>perturbAmplitude</code>	Real number	Normalized <code>Phi</code> amplitude of the initial Gaussian perturbation.	The perturbation is proportional to <code>amplitude * exp[-(X - Xc)^2 / (2 * width^2)]</code> , where <code>Xc = (radialIndex - 1) / (gridNx - 1)</code> .	<code>const mhdReal perturbAmplitude = 2.5e-9;</code>

Filtering

Parameter	Allowed Values	Description	Notes	Example
<code>ifFilterN_<MHDfield></code>	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the MHD field.		<code>using ifFilterN_Phi = trueType;</code>
<code>ifFilterN_dP</code>	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifFilterN_dP = trueType;</code>
<code>removeN_<MHDfield></code>	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the MHD field.		<code>constexpr std::array<int, 1> removeN_Phi = {7};</code>
<code>removeN_dP</code>	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<int, 0> removeN_dP = {};</code>
<code>selectNM_<MHDfield></code>	<code>{{{N, leftM, rightM}, ...}}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the MHD field.		<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{{0, 0, 0}}};</code>
<code>selectNM_dP</code>	<code>{{{N, leftM, rightM}, ...}}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{{0, 0, 1}}};</code>

MHD Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>ifNonlinearMHD</code>	<code>trueType</code> / <code>falseType</code>	Whether to include MHD nonlinear terms.		<code>using ifNonlinearMHD = trueType;</code>
<code>ifEparallel</code>	<code>trueType</code> / <code>falseType</code>	Whether to include the parallel electric-field term.		<code>using ifEparallel = trueType;</code>
<code>ifFLRMHD</code>	<code>trueType</code> / <code>falseType</code>	Whether to include the FLR effect from background thermal ions in the Poisson equation.		<code>using ifFLRMHD = falseType;</code>
<code>ifQNeutrality</code>	<code>trueType</code> / <code>falseType</code>	Whether to compute the electron density perturbation from quasi-neutrality.	When enabled, <code>dNe</code> is determined by vorticity and the ion density perturbation.	<code>using ifQNeutrality = trueType;</code>
<code>ifReducedMHD</code>	<code>trueType</code> / <code>falseType</code>	Evolve the pressure of the background thermal plasma with a single-fluid pressure equation; disables evolution of <code>dNe</code> and <code>dTe</code> .	Use with <code>ifEparallel=falseType</code> .	<code>using ifReducedMHD = falseType;</code>
<code>Gamma</code>	Real number	Adiabatic index in the single-fluid pressure equation.		<code>const mhdReal Gamma = 0.0;</code>
<code>ifMaxwellStress</code>	<code>trueType</code> / <code>falseType</code>	Whether to include Maxwell stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifMaxwellStress = trueType;</code>
<code>ifReynoldsStress</code>	<code>trueType</code> / <code>falseType</code>	Whether to include Reynolds stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifReynoldsStress = trueType;</code>
<code>MaxwellStressCoef</code>	Real number	Maxwell stress coefficient.	Effective only when <code>ifMaxwellStress=trueType</code> .	<code>const mhdReal MaxwellStressCoef = 1.0;</code>
<code>ReynoldsStressCoef</code>	Real number	Reynolds stress coefficient.	Effective only when <code>ifReynoldsStress=trueType</code> .	<code>const mhdReal ReynoldsStressCoef = 1.0;</code>

Dissipation

Parameter	Allowed Values	Description	Notes	Example
<code>ifNablaPerp2<MHDfield></code>	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular MHD-field dissipation.		<code>using ifNablaPerp2Phi = trueType;</code>
<code>perp2<MHDfield></code>	Real number	Coefficient for second-order perpendicular MHD-field dissipation.		<code>const mhdReal perp2Phi = 1.0e-7;</code>
<code>ifNablaPara4<MHDfield></code>	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel MHD-field dissipation.		<code>using ifNablaPara4Phi = trueType;</code>
<code>para4<MHDfield></code>	Real number	Coefficient for fourth-order parallel MHD-field dissipation.		<code>const mhdReal para4Phi = 1.0e-6;</code>
<code>ifNablaPerp2dP</code>	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPerp2dP = trueType;</code>
<code>perp2dP</code>	Real number	Coefficient for second-order perpendicular PIC-pressure dissipation.		<code>const mhdReal perp2dP = 1.0e-6;</code>
<code>ifNablaPara4dP</code>	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPara4dP = falseType;</code>
<code>para4dP</code>	Real number	Coefficient for fourth-order parallel PIC-pressure dissipation.		<code>const mhdReal para4dP = 0.0;</code>

PIC Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>ifFLRPIC</code>	<code>trueType</code> / <code>falseType</code>	Whether to enable the FLR effect in the PIC component.		<code>using ifFLRPIC = trueType;</code>
<code>ifNonlinearPIC</code>	<code>trueType</code> / <code>falseType</code>	Whether to enable PIC nonlinear terms.		<code>using ifNonlinearPIC = trueType;</code>
<code>gyroNums</code>	Integer	Number of gyro-average points.	Must satisfy <code>gyroNums = 2^n</code> with <code>n >= 2</code> .	<code>const int gyroNums = 4;</code>

PIC Species Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>if<Species></code>	<code>trueType</code> / <code>falseType</code>	Whether to enable the corresponding ion species.		<code>using ifIon = trueType;</code>
<code><Species>Type</code>	<code>Maxwell</code> / <code>Slowing0</code> / <code>Slowing1</code> / <code>Slowing2</code> / <code>Slowing3</code>	Velocity distribution type.		<code>const distType IonType = Maxwell;</code>
<code><Species>Space</code>	<code>spaceReal</code> / <code>spaceUniform</code>	Marker-position sampling: physical spatial distribution or uniform.		<code>const spaceType IonSpace = spaceReal;</code>
<code><Species>Velocity</code>	<code>velocityReal</code> / <code>velocityUniform</code>	Marker-velocity sampling: physical velocity distribution or uniform.		<code>const velocityType IonVelocity = velocityUniform;</code>
<code><Species>Mass</code>	Real number	Ion mass, normalized by the proton mass.		<code>const picReal IonMass = 2.5;</code>
<code><Species>Char</code>	Real number	Ion charge, normalized by the electron charge.		<code>const picReal IonChar = 1.0;</code>
<code><Species>Beta</code>	Real number	On-axis ion beta $(P/(B0^2/(2*\mu0)))$.		<code>const picReal IonBeta = 0.037793721898356;</code>
<code><Species>Vmin</code>	Real number	Lower bound of the ion velocity, normalized by <code>VA0</code> .		<code>const picReal IonVmin = 0.0135;</code>
<code><Species>Vmax</code>	Real number	Upper bound of the ion velocity, normalized by <code>VA0</code> .		<code>const picReal IonVmax = 0.54;</code>
<code><Species>Vb</code>	Real number	Cutoff velocity of the slowing-down distribution, normalized by <code>VA0</code> .		<code>const picReal BeamVb = 0.1;</code>
<code><Species>DeltaV</code>	Real number	Cutoff width of the slowing-down distribution, normalized by <code>VA0</code> .		<code>const picReal BeamDeltaV = 0.1;</code>
<code><Species>Lambda0</code>	Real number	Center of <code>Lambda</code> in the anisotropic distribution. <code>Lambda = mu*B0/E</code> .		<code>const picReal BeamLambda0 = 0.4;</code>
<code><Species>DeltaLambda2</code>	Real number	Square of the <code>Lambda</code> width in the anisotropic distribution. <code>Lambda = mu*B0/E</code> .		<code>const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);</code>

Distribution Type	Formula
<code>Maxwell</code>	<code>f ~ n * T^(-3/2) * exp[-m*V^2/(2*T)]</code>
<code>Slowing0</code>	<code>f ~ n / (V^3 + Vc^3)</code>
<code>Slowing1</code>	<code>f ~ n * [1 + erf((Vb - V)/DeltaV)] / (V^3 + Vc^3)</code>
<code>Slowing2</code>	<code>f ~ n * exp[-(Lambda - Lambda0)^2 / DeltaLambda^2] / (V^3 + Vc^3)</code>
<code>Slowing3</code>	<code>f ~ n * [1 + erf((Vb - V)/DeltaV)] * exp[-(Lambda - Lambda0)^2 / DeltaLambda^2] / (V^3 + Vc^3)</code>

Diagnostics

Parameter	Allowed Values	Description	Notes	Example
<code>ifDiagAmplitude</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode amplitude.		<code>using ifDiagAmplitude = trueType;</code>
<code>ifDiagFrequency</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode frequency.		<code>using ifDiagFrequency = trueType;</code>
<code>ifDiagEparallel</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the parallel electric field.		<code>using ifDiagEparallel = trueType;</code>
<code>ifDiagDensity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the ion density perturbation.		<code>using ifDiagDensity = trueType;</code>
<code>ifDiagDiffusivity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose ion diffusivity.		<code>using ifDiagDiffusivity = trueType;</code>
<code>ifDiagZFDrive</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the zonal-flow drive source.		<code>using ifDiagZFDrive = falseType;</code>
<code>ifCheckNAN</code>	<code>trueType</code> / <code>falseType</code>	Whether to check for NaN.	If NaN is diagnosed, the program stops immediately and writes output.	<code>using ifCheckNAN = trueType;</code>

MHD Field Output

Parameter	Allowed Values	Description	Notes	Example
<code>ifOutputPhi</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>Phi</code> at multiple time points.		<code>using ifOutputPhi = trueType;</code>
<code>ifOutputA</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>A</code> at multiple time points.		<code>using ifOutputA = falseType;</code>
<code>ifOutputdNe</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dNe</code> at multiple time points.		<code>using ifOutputdNe = falseType;</code>
<code>ifOutputdTe</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dTe</code> at multiple time points.		<code>using ifOutputdTe = falseType;</code>
<code>ifOutputdP</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the single-fluid pressure <code>dP</code> at multiple time points.		<code>using ifOutputdP = falseType;</code>
<code>ifOutputdPi</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPi</code> at multiple time points.		<code>using ifOutputdPi = falseType;</code>
<code>ifOutputdPa</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPa</code> at multiple time points.		<code>using ifOutputdPa = falseType;</code>
<code>ifOutputdPb</code>	<code>trueType</code> / <code>falseType</code>	Whether to output <code>dPb</code> at multiple time points.		<code>using ifOutputdPb = falseType;</code>

Phase-space Output

Parameter	Allowed Values	Description	Notes	Example
<code>gridE</code>	Integer	Number of phase-space grid points in the <code>E</code> direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	<code>const int gridE = 96;</code>
<code>gridPphi</code>	Integer	Number of phase-space grid points in the <code>Pphi</code> direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	<code>const int gridPphi = 128;</code>
<code>gridLambda</code>	Integer	Number of phase-space grid points in the <code>Lambda</code> direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	<code>const int gridLambda = 48;</code>
<code>ppcPhase</code>	Integer	Average number of particles per species in each phase-space grid cell.		<code>const int ppcPhase = 2048;</code>
<code>ifOutputPhaseSpaceJacobian</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space Jacobian.		<code>using ifOutputPhaseSpaceJacobian = trueType;</code>
<code>ifOutputPhaseSpaceOrbit</code>	<code>trueType</code> / <code>falseType</code>	Whether to output phase-space orbit frequencies.		<code>using ifOutputPhaseSpaceOrbit = falseType;</code>
<code>ifOutputPhaseSpaceF0</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space equilibrium distribution function.		<code>using ifOutputPhaseSpaceF0 = falseType;</code>
<code>ifOutputPhaseSpaceDeltaF</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the phase-space perturbed distribution function.		<code>using ifOutputPhaseSpaceDeltaF = falseType;</code>
<code>ifOutputPhaseSpacePower</code>	<code>trueType</code> / <code>falseType</code>	Whether to output phase-space wave-particle interaction power.		<code>using ifOutputPhaseSpacePower = falseType;</code>
<code>gridVpara</code>	Integer	Number of pitch-space grid points in the <code>vpara</code> direction.		<code>const int gridVpara = 128;</code>
<code>gridVperp</code>	Integer	Number of pitch-space grid points in the <code>vperp</code> direction.		<code>const int gridVperp = 64;</code>
<code>ppcPitch</code>	Integer	Average number of particles per species in each pitch-space grid cell.		<code>const int ppcPitch = 2048;</code>
<code>ifOutputPitchSpaceJacobian</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space Jacobian.		<code>using ifOutputPitchSpaceJacobian = trueType;</code>
<code>ifOutputPitchSpaceF0</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space equilibrium distribution function.		<code>using ifOutputPitchSpaceF0 = falseType;</code>
<code>ifOutputPitchSpaceDeltaF</code>	<code>trueType</code> / <code>falseType</code>	Whether to output the pitch-space perturbed distribution function.		<code>using ifOutputPitchSpaceDeltaF = falseType;</code>
<code>ifOutputPitchSpacePower</code>	<code>trueType</code> / <code>falseType</code>	Whether to output pitch-space wave-particle interaction power.		<code>using ifOutputPitchSpacePower = falseType;</code>

Time Advancement

Parameter	Allowed Values	Description	Notes	Example
<code>ifContinue</code>	<code>trueType</code> / <code>falseType</code>	Whether to start from continue files.	The <code>inputDir</code> directory must contain the <code>MHDContinue</code> and <code>PICContinue</code> continue files. They come from the previous run's <code>outputDir/final</code> .	<code>using ifContinue = falseType;</code>
<code>continueSteps</code>	Integer	Number of steps already completed before resuming.	Must match the suffix of the continue file names.	<code>const int continueSteps = 0;</code>
<code>dt</code>	Real number	MHD time step.	Normalized by the Alfvén time <code>L0/VA0</code> .	<code>const double dt = 0.02;</code>
<code>totalSteps</code>	Integer	Total number of MHD steps in this run.		<code>const int totalSteps = 20000;</code>
<code>ratioDt</code>	Integer	Ratio of the PIC time step to the MHD time step.	PIC uses <code>dt * ratioDt</code> for each push.	<code>const int ratioDt = 1;</code>
<code>sortSteps</code>	Integer	Particle sorting interval.		<code>const int sortSteps = 25;</code>
<code>diagSteps</code>	Integer	Diagnostic sampling interval.		<code>const int diagSteps = 1;</code>
<code>outputSteps</code>	Integer	Field and phase-space output interval.		<code>const int outputSteps = 2500;</code>